Guía 2 · Procedimiento explicado

La misma construcción, con el motivo y la función técnica de cada paso.

---

1. Resultado y estructura

Tres proyectos independientes: un orquestador y dos aplicaciones Angular. El desarrollo se sirve desde los puertos 9000, 4201 y 4202.

```
spa5/
  package.json            # arranque conjunto
  root-config/            # single-spa + Vite, :9000
  news-app/               # Angular 20, :4201
  favorites-app/          # Angular 20, :4202
```

Por qué: el root-config decide qué microfrontend debe estar activo. Cada aplicación Angular conserva su proyecto, sus dependencias y su compilación. El navegador importa sus módulos ESM; no interviene Module Federation.

2. Node.js y directorio de trabajo

```
nvm use 22
node -v
npm -v
mkdir spa5
cd spa5
```

Por qué: Node ejecuta las herramientas de desarrollo. Los módulos ESM finales de los microfrontends se cargan en el navegador. La versión 22 fue la utilizada en las pruebas. Si la carpeta ya existe, omitir mkdir.

3. Root-config sin Webpack

```
mkdir root-config
cd root-config
npm init -y
npm pkg set type=module
npm install single-spa single-spa-layout
npm install zone.js@~0.15.0
npm install -D vite@6
npm pkg set "scripts.start=vite --host 127.0.0.1 --port 9000"
```

Por qué: single-spa gestiona el registro y los ciclos de vida. Zone.js se cargará una vez desde la aplicación raíz. Vite sirve el HTML y resuelve las importaciones durante el desarrollo. single-spa-layout queda instalado, aunque esta versión del ejercicio utiliza registro programático y no el Layout Engine.

root-config/src/main.js

```
import 'zone.js';
import { registerApplication, start } from 'single-spa';
import * as singleSpa from 'single-spa';

window.singleSpa = singleSpa; // Solo para depurar

registerApplication({
  name: '@vm/news-app',
  app: () => import('http://127.0.0.1:4201/main.js'),
  activeWhen: location =>
    location.pathname === '/news' ||
    location.pathname.startsWith('/news/')
});

registerApplication({
  name: '@vm/favorites-app',
  app: () => import('http://127.0.0.1:4202/main.js'),
  activeWhen: location =>
    location.pathname === '/favorites' ||
    location.pathname.startsWith('/favorites/')
});

start();
```

Por qué: app devuelve una promesa que resuelve un módulo con las funciones bootstrap, mount y unmount. activeWhen selecciona la aplicación por URL. start() inicia su orquestación. La variable de window es un atajo de diagnóstico, no un requisito de producción.

root-config/index.html (en la raíz, no en src)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Noticias y Favoritos - single-spa</title>
</head>
<body>
  <h1>Orquestador single-spa</h1>
  <nav>
    <a href="/news">Noticias</a>
    <a href="/favorites">Favoritos</a>
  </nav>
  <main>
    <div id="news-container"></div>
    <div id="favorites-container"></div>
  </main>
  <script type="module" src="/src/main.js"></script>
</body>
</html>
```

Por qué: Vite busca index.html en la raíz del proyecto. type="module" permite usar importaciones ESM. Los dos contenedores son ubicaciones físicas que permanecen en el DOM mientras single-spa monta o desmonta las aplicaciones Angular.

4. Crear news-app

```
cd ..
npx -p @angular/cli@20 ng new news-app --routing --style=css --skip-git
cd news-app
```

En el asistente de Angular responder No a SSR/SSG y No a «zoneless». Mantener el componente standalone y el builder @angular/build:application.

Por qué: SSR/SSG no se necesita en este ejercicio, y Zone.js se usa para la detección de cambios tradicional. No forzamos NgModules: single-spa puede gestionar también aplicaciones standalone.

news-app/src/main.single-spa.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { ApplicationRef } from '@angular/core';
import { appConfig } from './app/app.config';
import { App } from './app/app';

let angularApp: ApplicationRef | null = null;
let hostElement: HTMLElement | null = null;

export async function bootstrap() {
  console.log('News: bootstrap');
}

export async function mount() {
  console.log('News: mount');
  const container = document.getElementById('news-container');
  if (!container) throw new Error('Falta news-container');
  hostElement = document.createElement('app-root');
  container.appendChild(hostElement);
  try {
    angularApp = await bootstrapApplication(App, appConfig);
  } catch (error) {
    hostElement.remove();
    hostElement = null;
    throw error;
  }
}

export async function unmount() {
  console.log('News: unmount');
  angularApp?.destroy();
  angularApp = null;
  hostElement?.remove();
  hostElement = null;
}
```

Por qué: se mantiene main.ts para el arranque Angular habitual y se crea una entrada distinta para single-spa. bootstrap prepara una sola vez; mount crea el elemento raíz e inicializa Angular; unmount destruye ApplicationRef y retira el elemento. Es una integración manual y sencilla, no el adaptador single-spa-angular.

news-app/src/app/app.config.ts

Mantener los proveedores generados y añadir APP_BASE_HREF para la ruta /news. En nuestro proyecto el archivo queda así:

```
import { ApplicationConfig,
  provideBrowserGlobalErrorListeners,
  provideZoneChangeDetection } from '@angular/core';
import { APP_BASE_HREF } from '@angular/common';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
```

```
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    { provide: APP_BASE_HREF, useValue: '/news' }
  ]
};
```

Por qué: Angular Router necesita saber que la raíz de esta microaplicación es /news aunque la página completa esté servida por el orquestador en el puerto 9000. Se evita que el router interprete incorrectamente la URL o navegue a la raíz general.

news-app/angular.json

En projects.news-app.architect.build.configurations, añadir single-spa sin sustituir production ni development:

```
"single-spa": {
  "browser": "src/main.single-spa.ts",
  "outputHashing": "none",
  "optimization": false,
  "sourceMap": true
}
```

Conservar el builder @angular/build:application y la entrada predeterminada browser: src/main.ts.

Por qué: esta configuración alternativa hace que el build use la entrada con ciclos de vida sin perder la posibilidad de ejecutar Angular por separado. outputHashing: none deja una URL estable main.js; sourceMap facilita depurar; desactivar optimización simplifica la primera integración.

Compilar y verificar las exportaciones

```
npx ng build --configuration single-spa
Select-String -Path "dist/news-app/browser/main.js" -Pattern "export\s*\{"
```

Comprobar al final del archivo las exportaciones públicas bootstrap, mount y unmount.

Por qué: un bundle de aplicación puede generarse correctamente sin exponer la API que espera single-spa. La comprobación de exportaciones asegura que el archivo ESM puede ser importado y orquestado.

Vite para servir el resultado de Angular

```
npm install -D vite@6
npm pkg set "scripts.serve:micro=vite --host 127.0.0.1 --port 4201"
npm pkg set "scripts.build:watch=ng build --configuration single-spa --watch"
```

Crear news-app/vite.config.js:

```
const { defineConfig } = require('vite');
const path = require('node:path');

module.exports = defineConfig({
  root: path.resolve(__dirname, 'dist/news-app/browser'),
  server: {
    host: '127.0.0.1',
    port: 4201,
    strictPort: true,
    cors: true
  },
  appType: 'mpa'
});
```

Por qué: ng build realiza la compilación con esbuild. Aquí Vite solo sirve la carpeta compilada; no sustituye al builder Angular. cors: true habilita la importación desde el puerto 9000; strictPort evita que se cambie de puerto sin que el orquestador lo sepa. build:watch recompila al guardar archivos.

5. Crear favorites-app

```
cd ..
npx -p @angular/cli@20 ng new favorites-app --routing --style=css --skip-git
cd favorites-app
```

En el asistente de Angular responder No a SSR/SSG y No a «zoneless». Mantener el componente standalone y el builder @angular/build:application.

Por qué: SSR/SSG no se necesita en este ejercicio, y Zone.js se usa para la detección de cambios tradicional. No forzamos NgModules: single-spa puede gestionar también aplicaciones standalone.

favorites-app/src/main.single-spa.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { ApplicationRef } from '@angular/core';
import { appConfig } from './app/app.config';
import { App } from './app/app';

let angularApp: ApplicationRef | null = null;
let hostElement: HTMLElement | null = null;

export async function bootstrap() {
  console.log('Favorites: bootstrap');
}

export async function mount() {
  console.log('Favorites: mount');
  const container = document.getElementById('favorites-container');
  if (!container) throw new Error('Falta favorites-container');
  hostElement = document.createElement('app-root');
  container.appendChild(hostElement);
  try {
    angularApp = await bootstrapApplication(App, appConfig);
  } catch (error) {
    hostElement.remove();
    hostElement = null;
    throw error;
  }
}

export async function unmount() {
  console.log('Favorites: unmount');
  angularApp?.destroy();
  angularApp = null;
  hostElement?.remove();
  hostElement = null;
}
```

Por qué: se mantiene main.ts para el arranque Angular habitual y se crea una entrada distinta para single-spa. bootstrap prepara una sola vez; mount crea el elemento raíz e inicializa Angular; unmount destruye ApplicationRef y retira el elemento. Es una integración manual y sencilla, no el adaptador single-spa-angular.

favorites-app/src/app/app.config.ts

Mantener los proveedores generados y añadir APP_BASE_HREF para la ruta /favorites. En nuestro proyecto el archivo queda así:

```
import { ApplicationConfig,
  provideBrowserGlobalErrorListeners,
  provideZoneChangeDetection } from '@angular/core';
import { APP_BASE_HREF } from '@angular/common';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    { provide: APP_BASE_HREF, useValue: '/favorites' }
  ]
};
```

Por qué: Angular Router necesita saber que la raíz de esta microaplicación es /favorites aunque la página completa esté servida por el orquestador en el puerto 9000. Se evita que el router interprete incorrectamente la URL o navegue a la raíz general.

favorites-app/angular.json

En projects.favorites-app.architect.build.configurations, añadir single-spa sin sustituir production ni development:

```
"single-spa": {
  "browser": "src/main.single-spa.ts",
  "outputHashing": "none",
  "optimization": false,
  "sourceMap": true
}
```

Conservar el builder @angular/build:application y la entrada predeterminada browser: src/main.ts.

Por qué: esta configuración alternativa hace que el build use la entrada con ciclos de vida sin perder la posibilidad de ejecutar Angular por separado. outputHashing: none deja una URL estable main.js; sourceMap facilita depurar; desactivar optimización simplifica la primera integración.

Compilar y verificar las exportaciones

```
npx ng build --configuration single-spa
Select-String -Path "dist/favorites-app/browser/main.js" -Pattern "export\s*\{"
```

Comprobar al final del archivo las exportaciones públicas bootstrap, mount y unmount.

Por qué: un bundle de aplicación puede generarse correctamente sin exponer la API que espera single-spa. La comprobación de exportaciones asegura que el archivo ESM puede ser importado y orquestado.

Vite para servir el resultado de Angular

```
npm install -D vite@6
npm pkg set "scripts.serve:micro=vite --host 127.0.0.1 --port 4202"
npm pkg set "scripts.build:watch=ng build --configuration single-spa --watch"
```

Crear favorites-app/vite.config.js:

```
const { defineConfig } = require('vite');
```

```
const path = require('node:path');

module.exports = defineConfig({
  root: path.resolve(__dirname, 'dist/favorites-app/browser'),
  server: {
    host: '127.0.0.1',
    port: 4202,
    strictPort: true,
    cors: true
  },
  appType: 'mpa'
});
```

Por qué: ng build realiza la compilación con esbuild. Aquí Vite solo sirve la carpeta compilada; no sustituye al builder Angular. cors: true habilita la importación desde el puerto 9000; strictPort evita que se cambie de puerto sin que el orquestador lo sepa. build:watch recompila al guardar archivos.

6. Arranque de los cinco procesos

```
cd ..
npm init -y
npm install -D concurrently
```

En spa5/package.json, sustituir únicamente el valor de scripts por:

```
"scripts": {
  "start": "concurrently -k -n ROOT,NEWS-BUILD,NEWS,FAV-BUILD,FAVORITES \"npm start --prefix root-config\"
\"npm run build:watch --prefix news-app\" \"npm run serve:micro --prefix news-app\" \"npm run build:watch
--prefix favorites-app\" \"npm run serve:micro --prefix favorites-app\""
}

npm start
```

Arrancan: root-config (9000), compilación vigilada de Noticias, servidor de Noticias (4201), compilación vigilada de Favoritos y servidor de Favoritos (4202). Esperar a la primera compilación antes de abrir el navegador.

Por qué: concurrently mantiene los cinco comandos activos en una sola terminal. Las compilaciones --watch actualizan los archivos de dist; los servidores Vite publican ese resultado. -k detiene los restantes procesos si uno finaliza. No es recarga automática garantizada del navegador; si hace falta, pulsar Ctrl+F5.

7. Comprobación funcional

```
http://127.0.0.1:9000/news
http://127.0.0.1:9000/favorites
```

En la consola del navegador (F12):

```
window.singleSpa.getAppNames();
console.table(window.singleSpa.getAppNames().map(name => ({
  aplicacion: name,
  estado: window.singleSpa.getAppStatus(name)
})));
```

La aplicación seleccionada debe estar en MOUNTED. La otra puede estar en NOT_LOADED (todavía no visitada) o NOT_MOUNTED (cargada, pero desmontada). Navegar por los enlaces y comprobar ambas direcciones.

Por qué: esta prueba demuestra que single-spa reconoce las aplicaciones y ejecuta sus ciclos de vida. La importación de un main.js por sí sola no demuestra que Angular se monte o se destruya correctamente.

8. Trabajo diario y alcance de la guía

Desde spa5, ejecutar npm start. Para cambios de código Angular, esperar a build:watch y refrescar la página si fuese necesario. Detener todo con Ctrl+C.

Importante: este ejercicio se verificó como entorno de desarrollo. Para producción hay que preparar URLs y builds de despliegue, gestionar los estilos globales de cada Angular y evaluar el aislamiento de CSS, la carga de recursos, los errores y las rutas profundas. El servidor de la raíz utiliza Vite durante el desarrollo, y las URLs ESM apuntan a localhost.

9. Ideas clave y límites técnicos

• single-spa no compila Angular: registra aplicaciones y decide cuándo ejecutar sus ciclos de vida.

• esbuild genera el JavaScript: Angular usa su builder application y produce un módulo que exporta las funciones requeridas.

• Vite sirve archivos: aquí el root usa Vite para desarrollo y cada microfrontend sirve su carpeta dist.

• ESM no equivale a Module Federation: usamos import() por URL; no hay negociación automática de librerías compartidas.

• Zone.js sí es global: lo importa el root una vez; los polyfills.js de cada Angular no se importan por separado.

• Dependencias y estilos: cada bundle Angular puede incluir librerías repetidas y los styles.css generados no se cargan automáticamente al importar main.js.

• El código de ciclos de vida es deliberadamente mínimo: para un proyecto real habría que endurecer el montaje, el tratamiento de errores y las particularidades de Angular y su router.